Clicky for Windows — Product & Technical Blueprint

Research, architecture, migration strategy, MVP, and implementation roadmap
October 10, 2026

1. Executive Summary

Build a Windows-native AI desktop companion inspired by Clicky/HeyClicky. Preserve the core interaction model—voice input, screen understanding, AI reasoning, spoken response, and visual pointing—while replacing macOS-specific components with Windows APIs.

The original public Clicky repository is MIT-licensed. Newer HeyClicky development is not necessarily represented by the public repository, so the public code should be treated as the reference implementation rather than assumed to be the current private product code.

Recommended approach: inspect the original repository and existing Windows implementations, establish a working end-to-end MVP, then optimize reliability, latency, privacy, and coordinate accuracy before adding advanced agent features.

2. Product Problem and Solution

Clicky addresses the friction of traditional AI assistants: users must explain their current screen, switch applications, or translate textual instructions into UI actions.

Core workflow:

1. User holds a global shortcut.

2. Microphone captures the question.

3. Speech is transcribed.

4. The current screen is captured.

5. The AI receives the question plus screen context.

6. The AI produces an answer and, when appropriate, a structured screen target.

7. Text-to-speech speaks the answer.

8. A visual overlay points toward the relevant interface element.

Primary use cases include learning unfamiliar software, finding settings, understanding errors, receiving contextual instructions, and getting hands-free assistance while staying in the current application.

3. Original Technology Stack

The public Clicky implementation is primarily Swift/macOS technology:

• Swift — application logic

• SwiftUI — UI

• AppKit — native windows and overlays

• ScreenCaptureKit — screen capture

• AVAudioEngine — microphone/audio handling

• Core Graphics event APIs — global shortcut handling

• AssemblyAI — streaming speech recognition

• Anthropic Claude — AI reasoning/vision

• ElevenLabs — text-to-speech

• Cloudflare Workers — API proxy

• Server-Sent Events — streaming

• PostHog — analytics

SwiftUI is an Apple-platform framework. It cannot simply be compiled into a native Windows UI. The application behavior can be preserved while the OS-specific layer is replaced.

4. Core Architecture

User

↓

Global hotkey

↓

Microphone capture

↓

Speech recognition

↓

Question + screenshot

↓

AI vision/reasoning

↓

Structured response

■■■→ Text-to-speech

■■■→ Point coordinates → Windows overlay

The original code separates important responsibilities around a companion manager, screen capture, dictation/transcription providers, AI API access, speech synthesis, and an overlay window.

5. Pointing Protocol

The original implementation uses structured markers such as:

[POINT:x,y:label:screenN]

The AI determines the relevant location; the desktop application renders the visual guidance. The Windows implementation should preserve this separation and add coordinate validation.

Windows must account for multiple monitors, negative virtual-desktop coordinates, mixed DPI scaling, monitor rearrangement, invalid/out-of-bounds targets, and targets that disappear during an interaction.

6. Windows Stack

Recommended first implementation:

• C#

• .NET

• WPF for the desktop UI

• Windows Graphics Capture or an appropriate Windows capture API

• Win32 RegisterHotKey/global shortcut handling

• NAudio/Windows audio APIs

• Transparent topmost WPF overlay

• Pluggable AI provider interface

• Pluggable speech-to-text provider

• Pluggable text-to-speech provider

• Secure Windows credential storage

• xUnit/integration tests

Electron + TypeScript remains a viable alternative, particularly if cross-platform support becomes important.

7. macOS → Windows Migration

Swift → C#

SwiftUI → WPF/WinUI

AppKit → Windows windowing APIs

ScreenCaptureKit → Windows Graphics Capture

AVAudioEngine → Windows audio APIs/NAudio

Core Graphics event tap → Win32 global hotkeys

macOS overlay → transparent WPF/WinUI overlay

Claude/API logic → provider abstraction with compatible API behavior

Transcription/TTS → provider abstractions

Cloudflare Worker → reuse if its interface remains appropriate

[POINT:...] protocol → preserve and validate

The correct strategy is a port of the architecture and behavior, not a direct compilation of the macOS application.

8. Existing Windows References

Two useful reference implementations should be compared with the original source before starting a rewrite:

• C# / WPF: https://github.com/emreyilmaz46/clicky_windows

• Electron / TypeScript: https://github.com/tekram/clicky-windows

The C# project is particularly relevant for native Windows integration. Build and test existing implementations rather than assuming their README claims are fully verified.

9. Latency and Real-Time Requirement

Speed is a first-class product requirement. The assistant should feel conversational rather than like a screenshot-upload chatbot.

Design principles:

• Stream speech recognition where possible.

• Capture the screen with minimal overhead.

• Stream AI output rather than waiting unnecessarily for a full response.

• Begin TTS as soon as safe response chunks are available.

• Keep persistent connections where they materially reduce latency.

• Avoid unnecessary model hops.

• Instrument end-to-end latency.

Measure:

1. Hotkey → recording start

2. Speech → transcript availability

3. Transcript → AI first token

4. AI first token → first spoken audio

5. Total interaction latency

For initial experiments, use an OpenRouter-compatible endpoint or another low-cost/free inference provider if it offers a sufficiently fast vision-capable model. A realtime model/provider can later be introduced behind the same provider interface. Do not hard-code the architecture to one vendor.

10. MVP

The first complete version should support:

1. System tray application

2. Configurable global shortcut

3. Push-to-talk

4. Speech transcription

5. Screen capture

6. AI visual reasoning

7. Streaming/low-latency response

8. Spoken response

9. Floating cursor overlay

10. Structured pointing

11. Multi-monitor coordinate handling

12. Secure configuration

Acceptance criterion: the user can hold a key, ask a question about the current screen, receive a grounded answer quickly, hear it spoken, and see the assistant point toward the relevant location.

11. Reliability

Use an explicit state machine:

IDLE → LISTENING → TRANSCRIBING → THINKING → SPEAKING → IDLE

Handle:

• incomplete AI streams

• network errors

• timeouts

• cancellation

• microphone failures

• speech recognition failures

• TTS failures

• invalid model output

• invalid coordinates

• monitor/DPI changes

No failed or partial request should silently appear successful.

12. Privacy and Security

Because screenshots and audio may contain sensitive information:

• Prefer explicit push-to-talk.

• Make capture state visible.

• Avoid unnecessary screenshot retention.

• Protect API credentials using Windows-secure storage.

• Never log secrets.

• Provide cancellation.

• Document which external providers receive data.

• Consider application exclusions.

• Add optional local speech recognition.

A local speech engine does not make the entire application local if screenshots are still sent to a cloud AI provider.

13. Advanced Features — After MVP

Only after the core loop is stable:

• application-specific guidance

• step-by-step tutorials

• conversation history

• persistent memory

• local inference

• multiple AI providers

• computer-use actions

• external integrations

• specialized agents

Computer-use actions should be permission-controlled and require confirmation for sensitive or destructive operations.

14. Development Roadmap

Phase 1 — Research

Inspect the original source, licenses, Windows implementations, dependencies, and feature gaps.

Phase 2 — Core Windows interaction

Build tray app, global hotkey, audio capture, transcription, screenshot capture, AI request, TTS, and overlay.

Phase 3 — Reliability

Add streaming/error handling, cancellation, timeouts, multi-monitor/DPI support, coordinate validation, recovery, logging, and tests.

Phase 4 — Privacy/security

Secure credentials, minimize retention, explicit capture controls, provider transparency, and optional local speech.

Phase 5 — Advanced capabilities

Memory, application-specific knowledge, local models, computer-use actions, and integrations.

Phase 6 — Release

Installer, CI, automated tests, documentation, demo video, architecture document, and known-limitations list.

15. Licensing

The original public repository is MIT-licensed according to its published documentation. Before redistribution, retain applicable license/copyright notices and independently verify the licenses of every dependency and any code copied from other implementations.

Do not treat newer private HeyClicky code as open source merely because the original repository is public, and do not imply official endorsement by the original team.

16. Immediate Action Plan

1. Clone the original Clicky repository.
2. Clone the C# Windows implementation.
3. Clone the Electron Windows implementation.
4. Verify licenses.
5. Build the available Windows implementations.
6. Test push-to-talk, screen capture, AI response, TTS, and pointing.
7. Compare implementation details against the original source.
8. Produce a file-by-file migration matrix: reusable, adaptable, or rewrite.
9. Select the foundation to extend.
10. Build the end-to-end MVP.
11. Benchmark latency.
12. Optimize the interaction loop before adding advanced features.

17. Final Engineering Direction

Build a Windows-native implementation of Clicky's interaction model rather than a superficial UI clone.

The differentiators should be:

• low perceived latency

• reliable streaming

• accurate screen/coordinate handling

• robust recovery

• privacy-conscious capture

• clean separation between AI logic and Windows integration

• automated testing

• reproducible builds

The first milestone is deliberately simple: hold a key, ask a question about the screen, receive a fast answer, hear it spoken, and see the assistant point to the relevant location.